

NYAYANET

Antigravity Project Reference & Deployment Handoff

Repository: github.com/ankit07-techie/manipal-ps-3

Problem statement: National Legal Hackathon 2.0 — PS3: Consumer Data, Consent and the Right to Seek Redressal

Purpose

A single reference for Antigravity while updating README.md and preparing deployment. Captures product scope, architecture, legal-tech safeguards, verification status, Git constraints, and deployment plan. It is a handoff reference—not proof that every feature has been independently verified.

Current status: implementation and automated checks are reported complete; browser journey and final source/diff review remain outstanding. Latest audit reports no staging, commits, or pushes.

Prepared from the project context and the latest Antigravity verification report supplied in this conversation.

Contents

- 1. Product overview and user journey
- 2. Nine PS3 capabilities
- 3. Architecture and technology
- 4. Current implementation and data flow
- 5. Legal accuracy and responsible wording
- 6. Privacy, security, and deployment constraints
- 7. Verification status and known gaps
- 8. Git workflow and strict user-control rules
- 9. Deployment plan: GitHub → Render → Vercel
- 10. README checklist
- 11. Copy-paste prompt for Antigravity
- 12. Official references

Instructions for Antigravity

- Read this reference before editing README.md or deployment configuration.
- Inspect actual source and package scripts; this PDF does not replace repository inspection.
- Treat test results as reported evidence, not proof tests were rerun now.
- Never invent API routes, environment variable names, commands, features, or deployment settings.

1. Product overview and user journey

NyayaNet is a consumer-facing legal-tech prototype intended to help people understand privacy notices, inspect evidence, record consent-related preferences, and prepare redressal communications.

Problem statement: National Legal Hackathon 2.0, PS3 — Consumer Data, Consent and the Right to Seek Redressal.

Product principle: help consumers understand and act without pretending automated analysis is legal advice, an email draft has been sent, or a missing clause proves a legal violation.

Step	Goal	Expected experience
1	Understand	Upload/load a sample privacy notice; inspect categories and quoted evidence.
2	Evaluate	See what was found, what was not found in analysed text, and whether input was truncated.
3	Act	Prepare an editable inquiry, consent-withdrawal, or erasure-related draft with qualified legal context.
4	Record	Save a request locally and track status/timeline events.
5	Manage	Record a consumer preference and export request audit data.

Repository

github.com/ankit07-techie/manipal-ps-3

2. The nine PS3 capabilities

#	Requirement	Implementation intent / caveat
1	Simplify privacy notices	Analyze notices and present understandable findings with evidence.
2	Record/manage preferences	Persist local preference records; label them as local, not third-party enforcement.
3	Show collection, use, sharing, retention	Use categories such as collection, purpose, third-party sharing, and retention.
4	Raise requests/concerns	Editable request flow and mailto action; never claim email was sent automatically.
5	Maintain request/response records	Store request records and timeline events with statuses/timestamps.
6	Prepare complaints/communications	Generate editable, qualified self-advocacy drafts.
7	Create audit trail	Timeline and JSON export; do not call it cryptographically tamper-proof.
8	Identify further examination	Flag missing/unverified information and truncation as inquiry points, not proven violations.
9	Privacy/security by design	Bounded analysis, upload limits, security headers, rate limits; prototype lacks auth/isolation.

Latest report marks all nine PASS. Treat these as reported implementation claims until the manual browser journey and source review are completed.

3. Architecture and technology

Layer	Reported implementation
Frontend	React 19 + Vite + Tailwind CSS; TypeScript/TSX. Root: frontend/.
Backend	Node.js + Express + TypeScript API. Root: backend/.
Policy analysis	Gemini mode when configured, with deterministic fallback; bounded to first 30,000 characters with truncation disclosure.
Document ingestion	PDF, DOCX (paragraphs/tables), TXT paths reported; unsupported/corrupt inputs should get useful errors.
Evidence	Matcher checks quoted text against source; only call a quote verified if exact matching succeeds.
Persistence	File-backed JSON store at backend/data/nyayanet_store.json; atomic rename and serialized writes reported.
Legal registry	backend/src/data/legalSources.ts; official MeitY/Gazette links and accurate commencement statuses.
Tests/evaluation	backend/tests/runTests.ts; backend/tests/runEvaluation.ts and curated 24-example dataset.

Reported user-facing areas

- Dashboard
- Policy Analyzer
- Consent Manager / My Consent
- Redressal Studio / Raise Request
- Track Requests
- Knowledge Hub

Verify actual labels, API contracts, and npm scripts against source before documenting them as definitive.

4. Current implementation and data flow

Flow	Expected sequence
Policy analysis	Frontend submits file/text → backend extractor/validator → bounded analysis engine → findings/evidence → UI displays categories, exact quotes, uncertainty and truncation.
Drafting	User chooses concern type/context → draft generator creates editable communication with qualified legal grounds → user reviews and may open email client.
Request tracking	User saves request → storage adapter persists it → timeline records status changes/notes → tracker displays events and exports JSON.
Preferences	User changes preference → API records local preference/status → UI clarifies it is not automatically propagated to third parties.

Reported API routes include POST /api/analyze, POST /api/redressal/draft, GET/POST /api/redressal/requests, request event routes, GET/PATCH /api/preferences, and GET /api/health. Verify exact contracts in source before README publication.

5. Legal accuracy and responsible wording

The latest audit reports this phased commencement schedule under Gazette Notification G.S.R. 843(E), dated 13 November 2025, for the project's October 2026 context:

Group	Reported scope
Immediate: 13 Nov 2025	Sections 1(2), 2, 18–26, 35, 38–43, and 44(1) & (3).
12-month: 13 Nov 2026	Section 6(9) and Section 27(1)(d).
18-month: 13 May 2027	Sections 3–5, 6(1)–(8) and 6(10), 7–17, and 28–34.

This reproduces the latest audit's legal framing. Before making definitive claims in a public README or live product, verify against the official Gazette and current government sources.

Wording rules

- Distinguish provisions enacted/notified from provisions that have commenced.
- Do not claim a statutory breach solely because an item is missing from analysed text.
- Section 8(7) retention exceptions must not be omitted from retention-related drafts.
- Do not promise blanket deletion or a fixed 30-day timeline unless facts and legal basis support it.
- Drafts are self-advocacy assistance, not legal advice. Users must verify recipient details.
- A mailto link opens an email client; it does not send the message or prove delivery.

Official legal links

[MeitY — DPDP Act 2023](#)

[DPDP Act PDF](#)

[Official Gazette G.S.R. 843\(E\)](#)

[MeitY — DPDP Rules 2025](#)

6. Privacy, security, and deployment constraints

Concern	Constraint / action
Authentication / isolation	Not implemented per report. Do not expose publicly with real consumer data.
Storage	JSON file is for local/single-instance persistent-disk demo, not robust multi-user/serverless storage.
Ephemeral hosts	Local files can be lost when instances are recycled. Do not put backend JSON persistence on Vercel frontend.
Rate limits	Reported 30/min analysis and 40/min drafting; verify actual middleware and proxy/IP behavior.
CORS	Allow only the deployed frontend origin; don't use unrestricted CORS as a shortcut.
Uploads	Reported 10 MB in-memory upload limit; validate type/size and return safe errors.
Secrets	Keep keys in server-side host environment variables; never in VITE_ variables or committed files.
Runtime data	Exclude backend/data/ and consumer records from Git; confirm ignored and untracked.
Multi-user production	Add auth/authz and managed PostgreSQL (e.g. Supabase) with tenant scoping.

7. Verification status and known gaps

Check	Reported result / qualification
Backend tests	30 passed / 0 failed in latest report; rerun after any code/config changes.
Backend build	Reported successful TypeScript build.
Frontend build	Reported successful TypeScript/Vite build; 1,919 modules in latest report.
Benchmark	100% metrics on N=24 curated examples; not a universal accuracy claim.
Persistence restart	Synthetic request, two timeline events, and a preference reportedly survived a process restart.
Rate-limit bypass	Regression test reportedly checks client headers cannot bypass rate limiting.
Browser journey	INCOMPLETE. Browser automation failed due external model capacity; manual test remains.
Final source review	Report claims files inspected; human diff review remains prudent.
Git	Latest report: main branch, no staged changes, commits or pushes.

Manual browser journey

- Open frontend; check dashboard/navigation and console errors.
- Load sample policy, analyze it, inspect categories and evidence quotations.
- Test >30,000 characters and verify truncation warning/count.
- Carry a finding into the redressal form, generate a draft, inspect caveats/links.
- Save a request, inspect timeline, add an event/note, export JSON.
- Change a preference, refresh, and verify persistence.
- Check empty/error states, responsive layout, and deployed frontend-to-backend API.

8. Git workflow and strict user-control rules

Repository: github.com/ankit07-techie/manipal-ps-3. Branch reported: main, up to date with origin/main before deployment push. Changes were reported as unstaged.

MANDATORY: Never stage, commit, push, merge, rebase, reset, clean, or switch branches unless the user explicitly approves the relevant operation. A request to prepare deployment is not permission to perform all Git operations silently.

Before staging

```
git status --short
git diff --check
git diff --stat
git diff
```

Preserve existing .gitignore rules and ensure generated files/secrets are excluded:

```
**/node_modules/
**/dist/
**/.env
**/.env.*
!**/.env.example
backend/data/
```

Only after explicit approval, stage and inspect before commit:

```
git add -A
git diff --cached --check
git diff --cached --stat
git diff --cached
# Commit/push only after user approval.
```

Never commit consumer data, API keys, tokens, .env files, node_modules, or generated dist output unless there is an explicit intentional reason.

9. Deployment plan: GitHub → Render → Vercel

Recommended architecture: GitHub repository → Render Node/Express backend → Vercel static frontend. Frontend must use the deployed API URL; backend CORS must allow the exact frontend origin.

Step 1 — GitHub

- Review .gitignore and source diff; ensure no runtime JSON data or secrets are staged.
- After explicit user approval, stage, inspect staged diff, commit, and push to main.

Step 2 — Render backend

- Create a Render Web Service connected to the repository.
- Set Root Directory to backend if supported by repository layout.
- Confirm actual build/start scripts in backend/package.json; do not guess.
- Ensure Express listens on process.env.PORT and binds appropriately for hosting.
- Set server-side environment variables in Render; never hardcode secrets.
- Configure CORS for the exact deployed Vercel origin.
- Choose storage: persistent disk for limited single-instance demo, managed PostgreSQL for multi-user architecture.

Step 3 — Vercel frontend

- Import repository into Vercel; set Root Directory to frontend.
- Framework Vite; Build command npm run build; Output directory dist, subject to source verification.
- Discover the exact API-base variable from frontend source; do not invent its name.

- Set API base URL to the Render service URL; redeploy.

Step 4 — live smoke tests

- Check deployed frontend, browser console/network, backend health and CORS.
- Test analysis, draft generation, saving, event tracking, preference update, refresh and persistence.
- Confirm no calls to localhost and no secrets in built JavaScript or logs.

Hosting links

[Vercel](#) · [Vite deployment guide](#)

[Render](#) · [Render docs](#) · [Supabase](#)

Deployment safety gate

Do not deploy this unauthenticated single-tenant prototype for real consumer data. A persistent disk improves filesystem durability for a single instance; it does not provide authentication, tenant isolation, access control, or production-grade database semantics.

10. README checklist

- Project name, PS3 statement, concise value proposition.
- Feature list mapped to actual nine requirements.
- Architecture and directory layout.
- Screenshots from actual current UI only.
- Tech stack, local setup, verified commands.
- Environment variable names and placeholder `.env.example` only.
- Legal disclaimer and commencement caveat.
- Privacy/security limitations and synthetic-data warning.
- Deployment links only after successful deployment; separate frontend URL from backend API URL.
- Known limitations: benchmark N=24, browser verification status, no auth/isolation until addressed.
- No claims that mail is automatically sent, legal advice is provided, or missing clauses prove breach.

11. Copy-paste prompt for Antigravity

Read this PDF, “NyayaNet Antigravity Project Reference & Deployment Handoff”, and inspect the current repository before making changes.

Task: update README.md accurately for the existing NyayaNet project and prepare it for deployment.

1. Do not rebuild the app or refactor unrelated code.
2. Verify all scripts, API routes, environment-variable names, directory names, legal links and features against source. Never invent details.
3. Keep README professional and useful to hackathon judges/developers: overview, PS3 alignment, features, architecture, stack, local setup, verified environment variables, test/build commands, legal disclaimer, privacy/security limitations and deployment plan.
4. Do not claim browser E2E is complete; latest report says it is incomplete. Do not claim N=24's 100% benchmark proves general accuracy.
5. Explain preferences are local records and mailto opens an email client; it does not send mail automatically.
6. Disclose the unauthenticated single-tenant JSON prototype is unsuitable for real consumer data/public multi-user production.
7. Inspect `.gitignore`; report whether `backend/data/nyayanet_store.json`, `.env` files, `node_modules` and `dist` are excluded. Never include secrets or real user data.
8. Inspect frontend API configuration, backend scripts, PORT handling and CORS. Report exact settings for Vercel frontend + Render backend. Do not guess API variable names.
9. Preserve all existing work.
10. STRICT GIT RULE: do not run `git add`, `commit`, `push`, `merge`, `rebase`, `reset`, `clean`, or `switch` branches. Do not stage anything.

At the end report README summary, verified local commands, deployment environment variables discovered from source, `.gitignore` findings, blockers and whether manual browser testing is still needed. List changed files and stop.

12. Official references

- **GitHub repository:** <https://github.com/ankit07-techie/manipal-ps-3>
- **MeitY DPDP Act 2023:** <https://www.meity.gov.in/content/digital-personal-data-protection-act-2023-dpdp-act>
- **DPDP Act PDF:** <https://www.meity.gov.in/static/uploads/2024/02/Digital-Personal-Data-Protection-Act-2023-1.pdf>
- **Official Gazette G.S.R. 843(E):** <https://egazette.gov.in/WriteReadData/2025/267647.pdf>
- **MeitY DPDP Rules 2025:** <https://www.meity.gov.in/documents/act-and-policies/digital-personal-data-protection-rules-2025-gDOxUjMtQWa?pageTitle=Digital-Personal-Data-Protection-Rules-2025>
- **Vercel:** <https://vercel.com/>
- **Vite static deployment guide:** <https://vite.dev/guide/static-deploy>
- **Render:** <https://render.com/>

- **Render docs:** <https://render.com/docs>
- **Supabase:** <https://supabase.com/>

End of reference. The repository and verified runtime results remain the source of truth.